Coding for Artificial Intelligence

Problem Set 9

The Automated Meta-Package Inspector

AST Introspection, Descriptors and Advanced Function Signatures

Name: Tanishka Hira

Enrollment No.: A26MTAI0001

Course: Coding for Artificial Intelligence

Problem Set: 9

Assignment: The Automated Meta-Package Inspector

Date: 21 September 2026

School of Artificial Intelligence

1. Objective

The objective of this assignment is to develop a Python class named `SourceAuditor` that performs source-code inspection and basic source-distribution validation using Python's standard library.

The implementation makes use of the `ast`, `os`, and `re` modules.

The program performs the following tasks:

- Parses Python source code into an Abstract Syntax Tree.
- Checks spacing between top-level functions.
- Correctly handles decorated functions.
- Detects lines exceeding the maximum line length.
- Detects trailing whitespace.
- Detects missing spaces after type-annotation colons.
- Uses a property setter to validate a target path.
- Validates an sdist-style package directory.
- Performs a self-audit on its own source file.

2. Property Setter and Descriptor Pattern

The `target_path` attribute is validated using a property getter and an explicit setter.

```
@property
def target_path(self) -> str:
    return self._target_path

@target_path.setter
def target_path(self, value: str) -> None:
    if not os.path.exists(value):
        raise FileNotFoundError(
            f"Target path '{value}' does not exist."
        )

    self._target_path = value
```

If validation is performed only inside `__init__`, the path is checked only when the object is created.

A property setter, however, performs validation every time the attribute is assigned.

Therefore, even if `target_path` is changed later, the new value must still correspond to an existing path before it is stored.

3. Advanced Function Signatures

The implementation uses both positional-only and keyword-only parameters.

For example:

```
def audit_module_structure(
    self,
    file_contents: str,
    /,
    *,
    max_line_length: int = 79
) -> dict:
```

The symbol:

```
/
```

means that `file_contents` is positional-only.

The symbol:

```
*
```

means that `max_line_length` is keyword-only.

Therefore, the following call is valid:

```
auditor.audit_module_structure(
    source,
    max_line_length=79
)
```

The constructor and `verify_sdist_layout()` also use positional-only path parameters.

4. Requirement 1: AST-Based Function Separation

The source code is first parsed into an Abstract Syntax Tree using:

```
tree = ast.parse(file_contents)
```

Only functions defined directly at module level are selected.

```
top_level_functions = [
    node
    for node in tree.body
    if isinstance(node, ast.FunctionDef)
]
```

Functions nested inside classes or other functions are therefore not included in this check.

For every consecutive pair of top-level functions, the ending line number of the previous function and starting line number of the next function are examined.

```
for previous, current in zip(
    top_level_functions,
    top_level_functions[1:]
):
    previous_end = previous.end_lineno
```

The number of blank lines between them is calculated as:

```
blank_lines = current_start - previous_end - 1
```

PEP 8 requires at least two blank lines between top-level functions.
Therefore:

```
if blank_lines < 2:
    spacing_violations.append(current.name)
```

records the name of the second function whenever the spacing is insufficient.

5. Decorator and Line-Number Pitfall

A special case occurs when the second function has a decorator.
Consider:

```
def first_function():
    return 1


@staticmethod
def second_function():
    return 2
```

For a decorated function, `FunctionDef.lineno` refers to the line containing the `def` statement.

However, the decorator is actually the first source line belonging to the function.
Therefore, using only:

```
current.lineno
```

could incorrectly calculate the number of blank lines.
The implementation handles this using:

```
if current.decorator_list:
    current_start = current.decorator_list[0].lineno
else:
    current_start = current.lineno
```

When decorators are present, the first decorator line is used as the effective starting line.

6. Requirement 2: Line and Whitespace Rule Engine

6.1 Line-Length Validation

The source code is first divided into individual lines:

```
lines = file_contents.splitlines()
```

Each line is then checked against the maximum permitted line length.

```
for line_number, line in enumerate(lines, start=1):
    if len(line) > max_line_length:
        line_length_violations.append(line_number)
```

The default maximum value is:

```
79 characters
```

Only the corresponding line number is stored in the result.

6.2 Trailing Whitespace

Trailing spaces and tabs are detected using the regular expression:

```
r"[ \t]+$"
```

The complete condition is:

```
if line.strip() and re.search(r"[ \t]+$", line):
    trailing_whitespace_violations.append(
        line_number
    )
```

The expression `line.strip()` ensures that whitespace-only blank lines are ignored.

Therefore, a blank line containing spaces is not treated as a trailing-whitespace violation.

6.3 Type-Annotation Colon Spacing

The auditor also checks whether type-annotation colons are followed by whitespace.

For example:

```
def test(value:str):
    return value
```

contains a violation because no space occurs after the colon.

The correct syntax is:

```
def test(value: str):
    return value
```

The annotation location is obtained from the AST.

The source line is then searched backward to locate the associated colon:

```
colon_position = line.rfind(
    ":",
    0,
    annotation_column
)
```

The character immediately following the colon is checked:

```
if (
    colon_position + 1 < len(line)
    and not line[colon_position + 1].isspace()
):
    colon_spacing_violations.append(
        line_number
    )
```

Duplicate line numbers are removed using:

```
colon_spacing_violations = list(
    dict.fromkeys(
        colon_spacing_violations
    )
)
```

7. Requirement 3: Manifest and Distribution Validator

The method:

```
verify_sdist_layout(directory_path)
```

checks whether a directory satisfies the required source-distribution layout.
The package used for testing contains:

```
my_package/
|
|-- README.txt
|-- setup.py
`-- vsearch.py
```

The method first confirms that the supplied path represents a directory:

```
if not os.path.isdir(directory_path):
    return False
```

The required file paths are created using:

```
setup_path = os.path.join(
    directory_path,
    "setup.py"
)

readme_path = os.path.join(
    directory_path,
    "README.txt"
)
```

The program then checks that both files exist:

```
if not os.path.isfile(setup_path):
    return False
```

```

if not os.path.isfile(readme_path):
    return False
```

7.1 setup.py

The test package contains a `setup.py` file such as:

```
from setuptools import setup


setup(
    name="my_package",
    version="1.0",
    py_modules=["vsearch"]
)
```

The important part is:

```
py_modules=["vsearch"]
```

The auditor reads the `py_modules` list from the AST of `setup.py`.
It then expects a corresponding Python module named:

`vsearch.py`

in the same directory.

7.2 vsearch.py

The test module contains a simple function:

```
def search4letters(
    phrase: str,
    letters: str = "aeiou"
) -> set:
    """Return letters found in the supplied phrase."""

    return set(letters).intersection(
        set(phrase)
    )
```

Since the module exists and is listed in `py_modules`, the module validation succeeds.

7.3 Source Distribution Validation Result

The final package validation produced:

```
TEST 6 - sdist layout
Valid package: True
```

This confirms that:

1. `setup.py` exists.
2. `README.txt` exists.
3. `vsearch.py` exists.
4. `vsearch` appears in the `py_modules` list.

Therefore, `verify_sdist_layout()` correctly returns `True`.

8. Requirement 4: Self-Testing Suite

The final part of the program performs a self-audit of `package_inspector.py`.

```
if __name__ == "__main__":
    assert not SourceAuditor(
        __file__
    ).audit_module_structure(
        open(
            __file__,
            encoding="utf-8"
        ).read()
    )["spacing_violations"], (
        "Self-audit failed: "
        "spacing violations found."
    )

    print(
        "Self-audit passed: no top-level "
        "function spacing violations."
    )
```

The program creates a `SourceAuditor` using its own file path, reads its own source code, and checks the returned `spacing_violations` list.

The assertion succeeds only when the list is empty.

8.1 Self-Audit Output

The final execution produced:

```
Self-audit passed: no top-level function spacing violations.
```

Screenshot package_ inspector.png was not uploaded.

9. Testing

A separate test program was used to validate the different components of `SourceAuditor`.

Six tests were performed.

9.1 Test 1: Function Spacing

Two correctly separated top-level functions were audited.

The result was:

```
TEST 1 - Function spacing

{'spacing_violations': [],
 'line_length_violations': [],
 'trailing_whitespace_violations': [],
 'colon_spacing_violations': []}
```

No violation was reported because the functions contained sufficient blank-line separation.

9.2 Test 2: Colon Spacing

A function argument was deliberately written without a space after the type-annotation colon.

For example:

```
1 def test(value:str):
2     return value
```

The result was:

```
'colon_spacing_violations': [1]
```

Therefore, the auditor correctly identified line 1.

9.3 Test 3: Line Length

A deliberately long source line was tested.

The result was:

```
'line_length_violations': [2]
```

The auditor therefore correctly reported line 2.

9.4 Test 4: Trailing Whitespace

Visible text followed by spaces was included in the source.

The result was:

```
'trailing_whitespace_violations': [2]
```

The auditor correctly detected the trailing whitespace.

9.5 Test 5: Decorated Function

A decorated function was used to confirm that its starting position was taken from the decorator instead of the `def` line.

The final test output contained:

```
'spacing_violations': []
```

because the tested source contained sufficient spacing before the decorator.

9.6 Test 6: sdist Layout

The final package directory contained:

```
README.txt
setup.py
vsearch.py
```

The final result was:

```
TEST 6 - sdist layout
Valid package: True
```

Thus, the package validator successfully confirmed the source distribution layout.

10. Test Result Summary

Test	Purpose	Result
1	Function spacing	No spacing violations were detected.
2	Type-annotation colon spacing	Line 1 was correctly reported.
3	Maximum line length	Line 2 was correctly reported.
4	Trailing whitespace	Line 2 was correctly reported.
5	Decorated function	The decorator line was handled correctly.
6	sdist package layout	The package was successfully validated and returned `True`.

10.1 Final Test Output

Screenshot test_ inspector.png was not uploaded.

11. Edge Cases

11.1 Exactly One Blank Line Between Top-Level Functions

Consider:

```
def first():
    pass

def second():
    pass
```

Only one blank line occurs between the functions.
Since the implementation uses:

```
if blank_lines < 2:
    spacing_violations.append(
        current.name
    )
```

the second function is correctly reported as a spacing violation.

11.2 Decorated Function Following Another Function

Consider:

```
def first():
    pass

@staticmethod
def second():
    pass
```

The auditor must not calculate spacing from:

`def second():`

Instead, it begins at:

`@staticmethod`

using:

```
current.decorator_list[0].lineno
```

Therefore, the decorator does not artificially increase the number of blank lines.

11.3 Missing README.txt

If a package contains `setup.py` and its module but `README.txt` is missing:

```
if not os.path.isfile(readme_path):
    return False
```

causes the package validator to return `False`.

11.4 Whitespace-Only Blank Line

For a line containing only spaces:

```
line.strip()
```

returns an empty string.

Therefore:

```
if line.strip() and re.search(
    r"[ \t]+$",
    line
):
```

evaluates to false and the blank line is not incorrectly recorded as a trailing-whitespace violation.

12. Submission Note

One important issue encountered while using Python's AST is that the `lineno` value of a decorated `FunctionDef` refers to the line containing the `def` statement rather than the decorator appearing above it.

If this line number were used directly, the decorator could incorrectly be counted as part of the blank-line separation between two top-level functions.

My implementation first checks the function's `decorator_list`. When a decorator exists, the line number of the first decorator is used as the effective starting line.

For example, I tested a top-level function followed by a function decorated with `@staticmethod`. The spacing calculation therefore begins at the decorator rather than the `def` statement. This ensures that the actual source-code layout is measured correctly.

13. Conclusion

The Automated Meta-Package Inspector demonstrates the use of Python AST introspection, descriptors, regular expressions, advanced function signatures, and filesystem operations.

The final `SourceAuditor` implementation successfully:

- detects insufficient top-level function spacing,
- handles decorated functions,
- detects excessive line length,
- detects trailing whitespace,
- detects incorrect type-annotation colon spacing,
- validates paths using a property setter,
- validates an sdist-style package layout, and
- performs a successful self-audit.

The final testing confirms that the required functionality operates as expected.